


Język Python

**Wykład 02**

Fizyka Medyczna

13 października 2026

# Podstawowe typy danych

Jacek Gatlik

e-mail: jacek.gatlik@agh.edu.pl
www: https://galaxy.agh.edu.pl/~jgatlik/

Wydział Fizyki i Informatyki Stosowanej,
Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

# Plan wykładu

Łańcuchy znaków

Listy

Krotki

Słowniki

Zbiory

Przykłady

Wyrażenia regularne

Sekcja 1

# Łańcuchy znaków

# Łańcuchy znaków

Łańcuchy znakowe, zwane również łańcuchami tekstowymi lub po prostu tekstem, są jednym z podstawowych typów danych w Pythonie. Służą do reprezentowania i manipulacji sekwencjami znaków, co jest kluczowe w wielu dziedzinach informatyki, takich jak analiza tekstu, przetwarzanie danych czy interakcja z użytkownikiem. W Pythonie łańcuchy znakowe są **niemutowalne** (immutable), co oznacza, że po ich utworzeniu nie można zmienić ich zawartości.

# Zapisywanie tekstu w Pythonie

W Pythonie łańcuchy znakowe można definiować na kilka sposobów:
**Pojedynczy cudzysłów:**

```
s = 'To jest łańcuch znakowy.'
```

**Podwójny cudzysłów**

```
s = "To jest również łańcuch znakowy."
```

**Potrójne cudzysłowy (pojedyncze lub podwójne):**

```
s = '''To jest
wieloliniowy
łańcuch znakowy.'''
```

```
s = """To jest
również wieloliniowy
łańcuch znakowy."""
```

# Znaki specjalne i łańcuchy surowe

W łańcuchach znakowych można używać znaków specjalnych za pomocą sekwencji ucieczki:

- \n – nowa linia
- \t – tabulacja
- \\ – backslash
- \′ – pojedynczy cudzysłów
- \” – podwójny cudzysłów

```
1 s = "Linia 1\nLinia 2"
```

**Łańcuchy surowe (raw strings)**
Aby zignorować sekwencje ucieczki, można użyć prefiksu ***r***:

```
1 s = r"C:\nowy_folder\plik.txt"
```

# Indeksowanie i wycinki tekstu

**Indeksowanie** - Łańcuchy znakowe są sekwencjami, co pozwala na dostęp do poszczególnych znaków za pomocą indeksów:

```
s = "Python"
print(s[0])   # 'P'
print(s[-1])  # 'n'
```

**Wycinki (slicing)** - Można pobierać podłańcuchy za pomocą operatora wycinania:

```
s = "Python"
print(s[0:3])   # 'Pyt'
print(s[2:])    # 'thon'
print(s[:2])    # 'Py'
print(s[::2])   # 'Pto'
```

**Niezmienność łańcuchów znakowych** - Ponieważ łańcuchy są niezmienne, nie można modyfikować ich zawartości:

```
s = "Python"
s[0] = 'J'   # TypeError: 'str' object does not support item assignment
```

# Operacje na łańcuchach znakowych

**Konkatenacja** - Łączenie łańcuchów za pomocą operatora $+$:

```
s1 = "Hello"
s2 = "World"
s = s1 + " " + s2   # 'Hello World'
```

**Powielanie** - Powielanie łańcucha za pomocą operatora $*$:

```
s = "Ha" * 3   # 'HaHaHa'
```

**Sprawdzanie przynależności** - Użycie operatora ***in***:

```
"Py" in "Python"    # True
"py" in "Python"    # False
```

**Długość łańcucha** - Funkcja ***len()*** zwraca liczbę znaków w łańcuchu:

```
len("Python")   # 6
```

# Metody łańcuchów znakowych I

### Zmiana wielkości liter

- ***str.lower()*** – zamienia wszystkie znaki na małe litery.
- ***str.upper()*** – zamienia wszystkie znaki na wielkie litery.
- ***str.capitalize()*** – pierwsza litera na wielką, reszta na małe.
- ***str.title()*** – pierwsze litery każdego słowa na wielkie.
- ***str.swapcase()*** – zamienia wielkość liter na przeciwną.

```
s = "pYtHoN"
s.lower()       # 'python'
s.upper()       # 'PYTHON'
s.capitalize()  # 'Python'
s.title()       # 'Python'
s.swapcase()    # 'PyThOn'
```

### Usuwanie białych znaków

- ***str.strip()*** – usuwa białe znaki z początku i końca.

# Metody łańcuchów znakowych II

- ***str.lstrip()*** – usuwa białe znaki z początku.
- ***str.rstrip()*** – usuwa białe znaki z końca.

```
s = "   Python   "
s.strip()    # 'Python'
s.lstrip()   # 'Python   '
s.rstrip()   # '   Python'
```

**Zastępowanie i dzielenie**

- ***str.replace(old, new[, count])*** – zastępuje podłańcuch old na new.
- ***str.split(separator[, maxsplit])*** – dzieli łańcuch na listę podłańcuchów.
- ***str.join(iterable)*** – łączy łańcuchy z obiektu iterowalnego, używając separatora.

# Metody łańcuchów znakowych III

```
s = "Python jest wspaniały"
s.replace("wspaniały", "świetny")   # 'Python jest świetny'

words = s.split()   # ['Python', 'jest', 'wspaniały']
" ".join(words)     # 'Python jest wspaniały'
```

**Wyszukiwanie**

- ***str.find(sub[, start[, end]])*** – zwraca najniższy indeks wystąpienia sub lub -1 przy braku dopasowania.
- ***str.rfind(sub[, start[, end]])*** – zwraca najwyższy indeks wystąpienia sub lub -1 przy braku dopasowania.
- ***str.index(sub[, start[, end]])*** – jak find, ale rzuca ValueError, jeśli nie znajdzie.
- ***str.count(sub[, start[, end]])*** – liczy wystąpienia sub.

# Metody łańcuchów znakowych IV

```
s = "Python jest wspaniały"
s.find("jest")    # 7
s.count("a")      # 2
```

### Sprawdzanie zawartości

- ***str.isalpha()*** – czy wszystkie znaki są literami.
- ***str.isdigit()*** – czy wszystkie znaki są cyframi.
- ***str.isalnum()*** – czy wszystkie znaki są alfanumeryczne.
- ***str.isspace()*** – czy wszystkie znaki są białymi znakami.
- ***str.startswith(prefix[, start[, end]])*** – czy łańcuch zaczyna się od prefix.
- ***str.endswith(suffix[, start[, end]])*** – czy łańcuch kończy się na suffix.

```
s = "Python"
s.isalpha()          # True
s.startswith("Py")   # True
s.endswith("on")     # True
```

# Formatowanie wyników w tekście

**Operator %** - Starsza metoda formatowania:

```
name = "Alice"
age = 30
s = "Imię: %s, Wiek: %d" % (name, age)
```

**Metoda** *str.format()* - Elastyczniejsza i bardziej czytelna:

```
s = "Imię: {}, Wiek: {}".format(name, age)
s = "Wiek: {1}, Imię: {0}, Wiek: {1}".format(name, age)
s = "Imię: {name}, Wiek: {age}".format(name=name, age=age)
```

***F-strings*** **(od Pythona 3.6)** - Zwięzła metoda osadzania wartości w tekście:

```
s = f"Imię: {name}, Wiek: {age}"
```

Można w nich używać wyrażeń:

```
s = f"Za rok {name} będzie mieć {age + 1} lat."
```

# Przykłady: niemutowalność i błędy metod

```
1 sekwencja = "UCAGUUUGGUCC"
2 sekwencja[0] = "A"
```

# Przykłady: niemutowalność i błędy metod

```
1 sekwencja = "UCAGUUUGGUCC"
2 sekwencja[0] = "A"
```

```
TypeError: 'str' object does not support item assignment
```

# Przykłady: niemutowalność i błędy metod

```
1 sekwencja = "UCAGUUUGGUCC"
2 sekwencja[0] = "A"
```

```
TypeError: 'str' object does not support item assignment
```

```
1 print(sekwencja.find("Z"))
```

# Przykłady: niemutowalność i błędy metod

```
1 sekwencja = "UCAGUUUGGUCC"
2 sekwencja[0] = "A"
```

```
TypeError: 'str' object does not support item assignment
```

```
1 print(sekwencja.find("Z"))
```

```
-1
```

# Przykłady: niemutowalność i błędy metod

```
1 sekwencja = "UCAGUUUGGUCC"
2 sekwencja[0] = "A"
```

```
TypeError: 'str' object does not support item assignment
```

```
1 print(sekwencja.find("Z"))
```

```
-1
```

```
1 print(sekwencja.index("Z"))
```

# Przykłady: niemutowalność i błędy metod

```
1 sekwencja = "UCAGUUUGGUCC"
2 sekwencja[0] = "A"
```

```
TypeError: 'str' object does not support item assignment
```

```
1 print(sekwencja.find("Z"))
```

```
-1
```

```
1 print(sekwencja.index("Z"))
```

```
ValueError: substring not found
```

# Przykłady: niemutowalność i błędy metod

```
sekwencja = "UCAGUUUGGUCC"
sekwencja[0] = "A"
```

```
TypeError: 'str' object does not support item assignment
```

```
print(sekwencja.find("Z"))
```

```
-1
```

```
print(sekwencja.index("Z"))
```

```
ValueError: substring not found
```

```
liczba = 8
print(liczba.lower())
```

# Przykłady: niemutowalność i błędy metod

```
sekwencja = "UCAGUUUGGUCC"
sekwencja[0] = "A"
```

```
TypeError: 'str' object does not support item assignment
```

```
print(sekwencja.find("Z"))
```

```
-1
```

```
print(sekwencja.index("Z"))
```

```
ValueError: substring not found
```

```
liczba = 8
print(liczba.lower())
```

```
AttributeError: 'int' object has no attribute 'lower'
```

Sekcja 2

# Listy

# Listy

Listy są jednym z fundamentalnych typów danych w Pythonie, służącym do przechowywania sekwencji elementów. Są one uporządkowane, **mutowalne** (mutable) i mogą zawierać elementy dowolnego typu, w tym również inne listy. Ich elastyczność i wszechstronność czynią je niezastąpionymi w wielu zastosowaniach programistycznych.

# Tworzenie list

**Puste listy**
Pustą listę można utworzyć na dwa sposoby:

```
lista_pusta = []
# lub
lista_pusta = list()
```

**Listy z początkowymi elementami**
Listy mogą być inicjalizowane z zestawem elementów:

```
lista = [1, 'tekst', 3.14, True]
```

**Użycie wyrażeń listowych (list comprehensions)**
Wyrażenia listowe pozwalają na tworzenie list w sposób zwięzły:

```
kwadraty = [x**2 for x in range(10)]
```

**Konwersja innych typów na listy**
Można konwertować iterowalne obiekty na listy:

```
lista_znaków = list('Python')
```

# Dostęp do elementów listy

**Indeksowanie**
Elementy listy są indeksowane od zera:

```
pierwszy_element = lista[0]
```

**Wycinki (slicing)**
Można pobierać podlisty:

```
podlista = lista[1:3]  # elementy o indeksach 1 i 2
```

**Indeksy ujemne**
Pozwalają na dostęp od końca listy:

```
ostatni_element = lista[-1]
```

# Dodawanie elementów do listy

**Zmiana wartości elementów** Listy są zmienne:

```
lista[0] = 'nowa wartość'
```

**Dodawanie elementów**

- **list.append()** - dodaje element na końcu listy.

```
lista.append('nowy element')
```

- **list.extend()** - rozszerza listę o elementy innej sekwencji.

```
lista.extend([1, 2, 3])
```

- **list.insert()** - wstawia element na określoną pozycję.

```
lista.insert(1, 'wstawiony element')
```

# Usuwanie elementów z listy

### Usuwanie elementów

- **list.remove()** - usuwa pierwsze wystąpienie wartości.

```
lista = ['a', 'element', 'b', 'c', 'd']
lista.remove('element')
```

- **list.pop()** - usuwa element z określonej pozycji i zwraca go.

```
element = lista.pop(2)
```

- **del** - usuwa element lub zakres elementów.

```
del lista[0]
del lista[1:3]
```

# Operacje na listach

**Konkatenacja** Łączenie list:

```
lista1 = [1, 2]
lista2 = [3, 4]
lista_polaczona = lista1 + lista2
```

**Powielanie** Powielanie elementów:

```
lista_powielona = lista1 * 3
```

**Członkostwo** Sprawdzanie obecności elementu:

```
if 'element' in lista:
    print('Znaleziono element')
```

# Sortowanie i pozostałe metody list

- ***list.sort()*** – sortuje listę na miejscu.
- ***list.reverse()*** – odwraca kolejność elementów.
- ***list.index()*** – zwraca indeks pierwszego wystąpienia wartości.
- ***list.copy()*** – tworzy **płytką** kopię listy.
- ***list.count()*** – zlicza wystąpienia wartości.
- ***list.clear()*** – usuwa wszystkie elementy.

```
lista = ['b', 'element', 'a', 'element']
lista.sort()
lista.reverse()
indeks = lista.index('element')
nowa_lista = lista.copy()
liczba = lista.count('element')
lista.clear()
```

# Wyrażenia listowe i numerowanie

## Wyrażenia listowe

- Podstawowa składnia

```
nowa_lista = [wyrażenie for element in kolekcja]
```

- Warunki w wyrażeniach

```
parzyste = [x for x in range(10) if x % 2 == 0]
```

- Zagnieżdżone wyrażenia

```
macierz = [[i * j for j in range(5)] for i in range(5)]
```

## Iterowanie po listach

- Pętla *for*

```
for element in lista:
    print(element)
```

- Funkcja *enumerate()* - pozwala na dostęp do indeksów

```
for indeks, wartość in enumerate(lista):
    print(f'Indeks {indeks}: {wartość}')
```

# Kopia płytka i głęboka

```
lista1 = [[1, 2], [3, 4]]
lista2 = lista1.copy()
lista2[0][0] = 99
print(lista1)
print(lista2)
```

# Kopia płytka i głęboka

```python
lista1 = [[1, 2], [3, 4]]
lista2 = lista1.copy()
lista2[0][0] = 99
print(lista1)
print(lista2)
```

```
[[99, 2], [3, 4]]
[[99, 2], [3, 4]]
```

# Kopia płytka i głęboka

```python
lista1 = [[1, 2], [3, 4]]
lista2 = lista1.copy()
lista2[0][0] = 99
print(lista1)
print(lista2)
```

```
[[99, 2], [3, 4]]
[[99, 2], [3, 4]]
```

```python
import copy
lista1 = [[1, 2], [3, 4]]
lista2 = copy.deepcopy(lista1)
lista2[0][0] = 99
print(lista1)
print(lista2)
```

```
[[1, 2], [3, 4]]
[[99, 2], [3, 4]]
```

Sekcja 3

# Krotki

# Krotki

Krotki (tuples) są jedną z podstawowych struktur danych w Pythonie. Są to uporządkowane, **niemutowalne** sekwencje, które mogą przechowywać elementy dowolnego typu. Po utworzeniu krotki nie można podmienić jej elementów, ale obiekty mutowalne wewnątrz niej mogą się zmieniać. Krotki są często używane do przechowywania danych heterogenicznych i mogą służyć jako klucze w słownikach, jeśli wszystkie ich elementy są hashowalne (obsługują funkcję `hash()`).

# Tworzenie krotek

### Puste krotki

Pustą krotkę można utworzyć na dwa sposoby:

```
krotka_pusta = ()
# lub
krotka_pusta = tuple()
```

### Krotki z elementami

Krotki mogą być inicjalizowane z zestawem elementów:

```
krotka = (1, 'tekst', 3.14, True)
```

### Krotki jednoelementowe

Aby utworzyć krotkę zawierającą tylko jeden element, konieczne jest użycie przecinka:

```
krotka_jednoelementowa = (5,)
```

Bez przecinka Python interpretuje nawiasy jako wyrażenie w nawiasach, a nie krotkę.

# Konwersja i pakowanie krotek

**Konwersja innych typów na krotki**
Można konwertować iterowalne obiekty na krotki:

```
1 krotka_znakow = tuple('Python')
```

**Krotki bez nawiasów (pakowanie krotek)**
Python pozwala na tworzenie krotek bez użycia nawiasów (pakowanie krotek):

```
1 krotka = 1, 2, 3
```

# Dostęp do elementów krotki

**Indeksowanie**
Elementy krotki są indeksowane od zera:

```
1 pierwszy_element = krotka[0]
```

**Wycinki (slicing)**
Można pobierać podkrotki:

```
1 podkrotka = krotka[1:3]  # elementy o indeksach 1 i 2
```

**Indeksy ujemne**
Pozwalają na dostęp od końca krotki:

```
1 ostatni_element = krotka[-1]
```

# Operacje na krotkach

**Konkatenacja**
Łączenie krotek:

```
krotka1 = (1, 2)
krotka2 = (3, 4)
krotka_połączona = krotka1 + krotka2
```

**Powielanie**
Powielanie elementów:

```
krotka_powielona = krotka1 * 3
```

**Członkostwo**
Sprawdzanie obecności elementu:

```
if 'element' in krotka:
    print('Znaleziono element')
```

# Metody krotek

Krotki mają ograniczony zestaw metod ze względu na ich niemutowalność.

- ***krotka.count()***: zlicza wystąpienia wartości.

```
krotka = ('a', 'element', 'element')
liczba = krotka.count('element')
```

- ***krotka.index()***: zwraca indeks pierwszego wystąpienia wartości.

```
indeks = krotka.index('element')
```

# Krotki a listy

**Główne różnice**

- Mutowalność: listy są mutowalne, krotki są niemutowalne.
- Składnia: listy używają nawiasów kwadratowych [], krotki używają nawiasów okrągłych () lub przecinków.
- Pamięć: w CPythonie krotki zwykle zajmują mniej pamięci niż listy z tymi samymi elementami.

**Kiedy używać krotek**

- Stałe dane: gdy dane nie powinny być zmieniane.
- Klucze w słownikach: krotki mogą być używane jako klucze, jeśli wszystkie ich elementy są hashowalne (obsługują funkcję `hash()`).
- Zwracanie wielu wartości z funkcji: funkcje mogą zwracać krotki zawierające wiele wartości.

# Rozpakowywanie krotek

**Przypisywanie wielu zmiennych**
Krotki umożliwiają przypisanie wielu zmiennych jednocześnie:

```
a, b, c = (1, 2, 3)
# lub
a, b, c = 1, 2, 3
```

**Zamiana wartości zmiennych**
Szybka zamiana wartości bez użycia zmiennej pomocniczej:

```
a, b = b, a
```

**Rozpakowywanie z gwiazdką ***
Pozwala na przypisanie reszty elementów do listy:

```
a, *b = (1, 2, 3, 4)
# a = 1, b = [2, 3, 4]
```

# Krotki z obiektami mutowalnymi

**Krotki zagnieżdżone**
Krotki mogą zawierać inne krotki lub listy:

```
krotka_zagnieżdżona = ((1, 2), (3, 4))
```

Dostęp do elementów:

```
element = krotka_zagnieżdżona[0][1]  # wartość 2
```

**Krotki zawierające obiekty mutowalne**
Chociaż krotki są niemutowalne, mogą zawierać obiekty mutowalne, takie jak listy:

```
krotka_z_listą = (1, [2, 3], 4)
```

Można modyfikować zawartość listy wewnątrz krotki:

```
krotka_z_listą[1].append(5)
# krotka_z_listą = (1, [2, 3, 5], 4)
```

# Zastosowania krotek

**Funkcje zwracające wiele wartości** - Krotki są często używane do zwracania wielu wartości z funkcji:

```
def podziel(a, b):
    return a // b, a % b

iloraz, reszta = podziel(10, 3)
```

**Klucze w słownikach** - Krotki mogą być używane jako klucze w słownikach:

```
słownik = {('a', 1): 'wartość'}
```

**Nazwane krotki** - Biblioteka standardowa Python oferuje `namedtuple`, która pozwala na tworzenie krotek z nazwanymi polami:

```
from collections import namedtuple

Punkt = namedtuple('Punkt', ['x', 'y'])
p = Punkt(1, 2)
print(p.x, p.y)
```

Sekcja 4

# Słowniki

# Słowniki

Słowniki (dictionaries) są jedną z fundamentalnych struktur danych w Pythonie, służącą do przechowywania par klucz-wartość. Są to **mutowalne** kolekcje zachowujące kolejność wstawiania kluczy (gwarantowane od Pythona 3.7), które pozwalają na szybki dostęp do wartości na podstawie kluczy. Klucze muszą być **hashowalne**, np. liczby, łańcuchy znaków lub krotki złożone z hashowalnych elementów, natomiast wartości mogą być dowolnego typu, w tym również innymi słownikami.

# Tworzenie słowników

### Puste słowniki

Pusty słownik można utworzyć na dwa sposoby:

```
słownik_pusty = {}
# lub
słownik_pusty = dict()
```

### Słowniki z elementami

Słowniki mogą być inicjalizowane z zestawem par klucz-wartość:

```
słownik = {'klucz1': 'wartość1', 'klucz2': 'wartość2'}
```

### Użycie konstruktora dict()

Możliwe jest tworzenie słowników za pomocą funkcji dict() z listy krotek:

```
słownik = dict([('klucz1', 'wartość1'), ('klucz2', 'wartość2')])
```

Lub z użyciem argumentów nazwanych:

```
słownik = dict(klucz1='wartość1', klucz2='wartość2')
```

# Dostęp do wartości słownika

**Pobieranie wartości na podstawie klucza**
Aby uzyskać wartość przypisaną do klucza:

```
wartość = słownik['klucz1']
```

Jeśli klucz nie istnieje, zostanie wygenerowany wyjątek `KeyError`. **Metoda get()**
Pozwala na bezpieczne pobieranie wartości:

```
wartość = słownik.get('klucz1')  # Zwróci None, jeśli klucz nie istnieje
# lub z wartością domyślną
wartość = słownik.get('klucz1', 'domyślna_wartość')
```

# Modyfikowanie słowników

### Dodawanie lub aktualizowanie elementów

Można dodawać nowe pary klucz-wartość lub aktualizować istniejące:

```
słownik['nowy_klucz'] = 'nowa_wartość'
słownik['klucz1'] = 'zaktualizowana_wartość'
```

### Usuwanie elementów

- ***del***: usuwa element o podanym kluczu.

```
del słownik['klucz1']
```

- ***słownik.pop()***: usuwa element i zwraca jego wartość.

```
wartość = słownik.pop('klucz2', 'domyślna_wartość')
```

- ***słownik.clear()***: usuwa wszystkie elementy ze słownika.

```
słownik.clear()
```

# Klucze, wartości i widoki słownika

### Sprawdzanie obecności klucza

```
if 'klucz1' in słownik:
    print('Klucz istnieje w słowniku')
```

### Widoki kluczy, wartości i par klucz-wartość

Metody zwracają widoki odzwierciedlające zmiany słownika. Listę można uzyskać np. przez `list(słownik.keys())`.

- ***słownik.keys()***: zwraca klucze.

```
klucze = słownik.keys()
```

- ***słownik.values()***: zwraca wartości.

```
wartości = słownik.values()
```

- ***słownik.items()***: zwraca pary klucz-wartość.

```
pary = słownik.items()
```

# Aktualizacja i kopiowanie słownika

- ***słownik.update()***: aktualizuje słownik o podane pary klucz-wartość.

```
słownik.update({'klucz3': 'wartość3', 'klucz4': 'wartość4'})
```

- ***słownik.setdefault()***: zwraca wartość dla klucza, jeśli klucz istnieje; w przeciwnym razie dodaje klucz z podaną wartością domyślną.

```
wartość = słownik.setdefault('klucz5', 'domyślna_wartość')
```

- ***słownik.copy()***: tworzy płytką kopię słownika.

```
nowy_słownik = słownik.copy()
```

# Iterowanie po słownikach

## Iterowanie po kluczach

```
for klucz in słownik:
    print(klucz)
```

Lub

```
for klucz in słownik.keys():
    print(klucz)
```

## Iterowanie po wartościach

```
for wartość in słownik.values():
    print(wartość)
```

## Iterowanie po parach klucz-wartość

```
for klucz, wartość in słownik.items():
    print(f'Klucz: {klucz}, Wartość: {wartość}')
```

# Słowniki zagnieżdżone

Słowniki mogą zawierać inne słowniki jako wartości:

```
słownik_zagnieżdżony = {
    'osoba1': {'imie': 'Jan', 'wiek': 30},
    'osoba2': {'imie': 'Anna', 'wiek': 25}
}
```

Dostęp do elementów:

```
wiek_osoby1 = słownik_zagnieżdżony['osoba1']['wiek']
```

Sekcja 5

# Zbiory

# Zbiory

Zbiory (sets) są wbudowaną strukturą danych w Pythonie, służącą do przechowywania **unikalnych**, **nieuporządkowanych** elementów. Są one mutowalne, co oznacza, że można dodawać i usuwać elementy po ich utworzeniu. Elementy zbioru muszą być **hashowalne**. Zbiory są szczególnie przydatne, gdy chcemy eliminować duplikaty lub wykonywać operacje teorii zbiorów, takie jak suma, przecięcie czy różnica.

# Tworzenie zbiorów

**Pusty zbiór**
Pusty zbiór tworzy się za pomocą funkcji set():

```
zbiór_pusty = set()
```

**Użycie {} utworzyłoby pusty słownik, nie zbiór! Zbiory z elementami**
Zbiór można utworzyć, umieszczając elementy w nawiasach klamrowych:

```
zbiór = {1, 2, 3, 4}
```

Lub konwertując iterowalny obiekt:

```
zbiór = set([1, 2, 3, 4])
```

**Główne właściwości**

- Unikalność elementów: każdy element w zbiorze jest unikalny.

```
zbiór = {1, 2, 2, 3}  # zbiór = {1, 2, 3}
```

- Nieuporządkowanie: elementy nie mają ustalonej kolejności, co oznacza, że nie można ich indeksować ani wycinać.

# Dodawanie i usuwanie elementów zbioru

**Dodawanie elementów**

- ***zbiór.add()***: dodaje pojedynczy element.

```
zbiór.add(5)
```

- ***zbiór.update()***: dodaje wiele elementów z innego iterowalnego obiektu.

```
zbiór.update([6, 7, 8])
```

**Usuwanie elementów**

- ***zbiór.remove()***: usuwa element; generuje `KeyError`, jeśli element nie istnieje.

```
zbiór.remove(2)
```

- ***zbiór.discard()***: usuwa element; nie generuje błędu, jeśli element nie istnieje.

```
zbiór.discard(10)
```

- ***zbiór.clear()***: usuwa wszystkie elementy.

```
zbiór.clear()
```

# Suma, część wspólna i różnica zbiorów

- Suma (| lub `union()`):

```
zbiór1 = {1, 2, 3}
zbiór2 = {3, 4, 5}
suma = zbiór1 | zbiór2  # {1, 2, 3, 4, 5}
```

- Przecięcie (& lub `intersection()`):

```
przecięcie = zbiór1 & zbiór2  # {3}
```

- Różnica (− lub `difference()`):

```
różnica = zbiór1 - zbiór2  # {1, 2}
```

- Różnica symetryczna (^ lub `symmetric_difference()`):

```
różnica_sym = zbiór1 ^ zbiór2  # {1, 2, 4, 5}
```

# Unikalne wartości i frozenset

## Sprawdzanie przynależności

```
if 3 in zbiór:
    print("Element jest w zbiorze")
```

## Zbiory niemutowalne (frozenset)

- ***frozenset***: niemutowalna wersja zbioru, której elementy nie mogą być zmieniane po utworzeniu.

```
zbiór_niemutowalny = frozenset([1, 2, 3])
```

- Przydatne jako klucze w słownikach lub elementy innych zbiorów.

## Zastosowania

- Eliminacja duplikatów:

```
lista = [1, 2, 2, 3, 4, 4]
unikalne = set(lista)  # {1, 2, 3, 4}
```

- Operacje zbiorowe: analiza danych, gdzie potrzebne są unikalne wartości lub porównania między zestawami danych.

Sekcja 6

# Przykłady

# Przypisanie a kopiowanie listy

```python
lista1 = [1, 2, 3]
lista2 = lista1
lista2.append(4)
print(lista1)
```

```
[1, 2, 3, 4]
```

# Przypisanie a kopiowanie listy

```python
lista1 = [1, 2, 3]
lista2 = lista1
lista2.append(4)
print(lista1)
```

```
[1, 2, 3, 4]
```

```python
lista1 = [1, 2, 3]
lista2 = lista1.copy()  # lub lista1[:]
lista2.append(4)
print(lista1)
print(lista2)
```

```
[1, 2, 3]
[1, 2, 3, 4]
```

# Krotki: lista w środku i ważny przecinek

Krotki są niemutowalne, ale mogą zawierać mutowalne obiekty, takie jak listy, które można modyfikować.

```
krotka = (1, [2, 3], 4)
krotka[1][0] = 99
print(krotka)
```

```
(1, [99, 3], 4)
```

# Krotki: lista w środku i ważny przecinek

Krotki są niemutowalne, ale mogą zawierać mutowalne obiekty, takie jak listy, które można modyfikować.

```
krotka = (1, [2, 3], 4)
krotka[1][0] = 99
print(krotka)
```

```
(1, [99, 3], 4)
```

```
krotka = (5)
print(type(krotka))

krotka = (5,)
print(type(krotka))
```

```
<class 'int'>
<class 'tuple'>
```

Sekcja 7

# Wyrażenia regularne

# Wyrażenia regularne i moduł re

Wyrażenia regularne wykorzystywane są do analizy wzorców w łańcuchach znaków.
Do pracy z nimi dedykowany jest moduł `re`.

Podstawowe funkcje oferowane przez moduł:

- ***re.compile()*** – zwraca obiekt wyrażenia regularnego, który może być używany z pozostałymi funkcjami,
- ***re.findall()*** – zwraca listę zawierającą wszystkie dopasowania,
- ***re.split()*** – zwraca listę zawierającą fragmenty ciągu znaków, podzielonego w miejscach dopasowania,
- ***re.sub()*** – domyślnie zastępuje wszystkie niepokrywające się dopasowania,
- ***re.search()*** – zwraca obiekt Match dla pierwszego dopasowania lub None, jeśli go nie ma.

Obiekt `Match` posiada własności oraz metody, służące do pobierania informacji o wyszukiwaniu oraz jego wyniku:

- *.string* – zwraca ciąg znaków przekazany do funkcji,
- *.group()* – zwraca fragment ciągu znaków zawierający dopasowanie,
- *.span()* – zwraca krotkę (początek, koniec), przy czym pozycja końcowa nie należy do dopasowania.

# Metaznaki i liczba powtórzeń

**Metaznaki:**

- ^ – początek ciągu znaków,
- $ – koniec ciągu lub pozycja przed końcowym znakiem nowej linii,
- . – domyślnie dowolny znak oprócz znaku nowej linii,
- ? – zero lub jedno wystąpienie poprzedniego elementu wzorca,
- * – zero lub więcej wystąpień poprzedniego elementu wzorca,
- + – jedno lub więcej wystąpień poprzedniego elementu wzorca,
- { } – określona liczba lub zakres powtórzeń poprzedniego elementu wzorca,
- [ ] – zbiór znaków,
- ( ) – grupa znaków,
- | – alternatywa.

# Zbiory i zakresy znaków

**Zbiory znaków:**

- *[abc]* – zwraca dopasowanie dla małych liter a, b lub c,
- *[a-n]* – zwraca dopasowanie dla małych liter od a do n,
- `[^abc]` – zwraca dopasowanie dla każdego znaku za wyjątkiem a, b lub c,
- *[123]* – zwraca dopasowanie dla cyfr 1, 2 lub 3,
- *[0-9]* – zwraca dopasowanie dla cyfr od 0 do 9,
- *[0-9][0-9]* – dopasowuje dwie kolejne cyfry, także wewnątrz dłuższej liczby,
- *[a-n][A-N]* – dopasowuje małą literę od a do n, a po niej wielką od A do N.

# Klasy znaków w wyrażeniach regularnych

**Klasy znaków:**

- \w – zwraca dopasowanie dla znaków alfanumerycznych lub podkreśleń,
- \W – zwraca dopasowanie dla znaków nie będących znakami alfanumerycznymi lub podkreśleniami,
- \d – zwraca dopasowanie dla dowolnej cyfry,
- \D – zwraca dopasowanie dla znaków nie będących cyframi,
- \s – dopasowuje białe znaki, np. spację, tabulację i znak nowej linii,
- \S – dopasowuje znaki niebędące białymi znakami.

# Dopasowania i dzielenie tekstu

```python
import re
t = "Ula przyjechała na turniej w golfa, ubrana w golf, golfem."
f = re.findall(r"golf\w*", t)
print(f)
```

# Dopasowania i dzielenie tekstu

```python
import re
t = "Ula przyjechała na turniej w golfa, ubrana w golf, golfem."
f = re.findall(r"golf\w*", t)
print(f)
```

```
['golfa', 'golf', 'golfem']
```

# Dopasowania i dzielenie tekstu

```python
import re
t = "Ula przyjechała na turniej w golfa, ubrana w golf, golfem."
f = re.findall(r"golf\w*", t)
print(f)
```

```
['golfa', 'golf', 'golfem']
```

```python
t = "W naszej szkole dzwonek na przerwę dzwoni o następujących godzinach: 08:45, 09.45, 10:45,
↪  11.45, 12:45, 13.45."
f = re.findall(r"(\d{2}:\d{2}|\d{2}\.\d{2})", t) # dwa sposoby zapisu godziny
print("Dzwonek na przerwę dzwoni o godzinach: ", f)
```

# Dopasowania i dzielenie tekstu

```python
import re
t = "Ula przyjechała na turniej w golfa, ubrana w golf, golfem."
f = re.findall(r"golf\w*", t)
print(f)
```

```
['golfa', 'golf', 'golfem']
```

```python
t = "W naszej szkole dzwonek na przerwę dzwoni o następujących godzinach: 08:45, 09.45, 10:45,
↪  11.45, 12:45, 13.45."
f = re.findall(r"(\d{2}:\d{2}|\d{2}\.\d{2})", t) # dwa sposoby zapisu godziny
print("Dzwonek na przerwę dzwoni o godzinach: ", f)
```

```
Dzwonek na przerwę dzwoni o godzinach: ['08:45', '09.45', '10:45', '11.45', '12:45',
'13.45']
```

# Dopasowania i dzielenie tekstu

```python
import re
t = "Ula przyjechała na turniej w golfa, ubrana w golf, golfem."
f = re.findall(r"golf\w*", t)
print(f)
```

```
['golfa', 'golf', 'golfem']
```

```python
t = "W naszej szkole dzwonek na przerwę dzwoni o następujących godzinach: 08:45, 09.45, 10:45,
↪  11.45, 12:45, 13.45."
f = re.findall(r"(\d{2}:\d{2}|\d{2}\.\d{2})", t) # dwa sposoby zapisu godziny
print("Dzwonek na przerwę dzwoni o godzinach: ", f)
```

```
Dzwonek na przerwę dzwoni o godzinach: ['08:45', '09.45', '10:45', '11.45', '12:45',
'13.45']
```

```python
t = "Tydzień ma 7 dni."
f = re.split(r"\s", t)
print(f)
```

# Dopasowania i dzielenie tekstu

```python
import re
t = "Ula przyjechała na turniej w golfa, ubrana w golf, golfem."
f = re.findall(r"golf\w*", t)
print(f)
```

```
['golfa', 'golf', 'golfem']
```

```python
t = "W naszej szkole dzwonek na przerwę dzwoni o następujących godzinach: 08:45, 09.45, 10:45,
↪  11.45, 12:45, 13.45."
f = re.findall(r"(\d{2}:\d{2}|\d{2}\.\d{2})", t) # dwa sposoby zapisu godziny
print("Dzwonek na przerwę dzwoni o godzinach: ", f)
```

```
Dzwonek na przerwę dzwoni o godzinach: ['08:45', '09.45', '10:45', '11.45', '12:45',
'13.45']
```

```python
t = "Tydzień ma 7 dni."
f = re.split(r"\s", t)
print(f)
```

```
['Tydzień', 'ma', '7', 'dni.']
```

# Wyszukiwanie i zamiana tekstu

```
t = "Dzisiaj jest poniedzialek. Jutro bedzie wtorek."
f = re.sub("wtorek", "piątek", t)
print(f)
```

# Wyszukiwanie i zamiana tekstu

```
1 t = "Dzisiaj jest poniedzialek. Jutro bedzie wtorek."
2 f = re.sub("wtorek", "piątek", t)
3 print(f)
```

```
Dzisiaj jest poniedzialek. Jutro bedzie piątek.
```

# Wyszukiwanie i zamiana tekstu

```python
t = "Dzisiaj jest poniedzialek. Jutro bedzie wtorek."
f = re.sub("wtorek", "piątek", t)
print(f)
```

```
Dzisiaj jest poniedzialek. Jutro bedzie piątek.
```

```python
t = "Mozesz do mnie zadzwonic. Oto moj numer telefonu: +48 123-456-789."
f = re.search(r"\+\d{1,3}\s\d{3}-\d{3}-\d{3}", t) # dowolny numer kierunkowy
print("Numer telefonu:", f.group())

f2 = re.search(r"^[A-Z][^.]*\.", t)
print("Pierwsze zdanie zaczyna się i kończy na pozycjach:", f2.span())
```

# Wyszukiwanie i zamiana tekstu

```python
t = "Dzisiaj jest poniedzialek. Jutro bedzie wtorek."
f = re.sub("wtorek", "piątek", t)
print(f)
```

```
Dzisiaj jest poniedzialek. Jutro bedzie piątek.
```

```python
t = "Mozesz do mnie zadzwonic. Oto moj numer telefonu: +48 123-456-789."
f = re.search(r"\+\d{1,3}\s\d{3}-\d{3}-\d{3}", t) # dowolny numer kierunkowy
print("Numer telefonu:", f.group())

f2 = re.search(r"^[A-Z][^.]*\.", t)
print("Pierwsze zdanie zaczyna się i kończy na pozycjach:", f2.span())
```

```
Numer telefonu: +48 123-456-789
Pierwsze zdanie zaczyna się i kończy na pozycjach: (0, 25)
```

# Wyodrębnianie liczb z tekstu

```python
import re

w = re.compile(r"\d+(?:\.\d+)?") # gdy używamy danego wyrażenia wielokrotnie
t = "Na dzisiejszym wykładzie obecnych jest 10 studentów. Tydzień temu zjawiło się ich 12, a dwa tygodnie temu - 19."
f =  re.findall(w, t)
print("Liczba studentów na wykładach: ", f)
t2 = "Dzisiejsza frekwencja wyniosła 52.63 %. Tydzień temu wyniosła ona 63.16 %, a dwa tygodnie temu - 100 %."
f2 =  re.findall(w, t2)
print("Frekwencje [%]: ", f2)
t3 = "Wszyscy zdali egzamin. Najwyższa ocena to 5.0, a najniższa to 3.5."
f3 =  re.findall(w, t3)
print("Najwyższa i najniższa ocena z egzaminu: ", f3)
```

# Wyodrębnianie liczb z tekstu

```python
import re

w = re.compile(r"\d+(?:\.\d+)?") # gdy używamy danego wyrażenia wielokrotnie
t = "Na dzisiejszym wykładzie obecnych jest 10 studentów. Tydzień temu zjawiło się ich 12, a dwa tygodnie temu - 19."
f =  re.findall(w, t)
print("Liczba studentów na wykładach: ", f)
t2 = "Dzisiejsza frekwencja wyniosła 52.63 %. Tydzień temu wyniosła ona 63.16 %, a dwa tygodnie temu - 100 %."
f2 =  re.findall(w, t2)
print("Frekwencje [%]: ", f2)
t3 = "Wszyscy zdali egzamin. Najwyższa ocena to 5.0, a najniższa to 3.5."
f3 =  re.findall(w, t3)
print("Najwyższa i najniższa ocena z egzaminu: ", f3)
```

```
Liczba studentów na wykładach: ['10', '12', '19']
Frekwencje [%]: ['52.63', '63.16', '100']
Najwyższa i najniższa ocena z egzaminu: ['5.0', '3.5']
```

# Dobór naturalny: opis modelu

**Zadanie (3 punkty)**

Proszę napisać program, który symuluje proces doboru naturalnego na poziomie genetycznym. Program będzie śledził zmiany frekwencji dwóch alleli jednego genu w populacji w kolejnych pokoleniach, uwzględniając dostosowanie trzech genotypów: AA, Aa oraz aa.

## Szczegółowy opis problemu:

**Genotypy i dostosowanie**

- **AA**: genotyp homozygotyczny dominujący.
- **Aa**: genotyp heterozygotyczny.
- **aa**: genotyp homozygotyczny recesywny.

Każdy genotyp ma przypisane dostosowanie $w$ (wartość między 0 a 1), które określa zdolność do przeżycia i reprodukcji. Wyższa wartość oznacza większą liczbę potomstwa.

**Frekwencje alleli**

- $p$: frekwencja allelu $A$ (wartość między 0 a 1).
- $q$: frekwencja allelu $a$ (gdzie $q = 1 - p$).

# Dobór naturalny: równania

**Prawo Hardy'ego-Weinberga** W dużej populacji panmiktycznej (losowo krzyżującej się), bez wpływu czynników ewolucyjnych, równowagowe częstości genotypów wynoszą:

$$P_{AA} = p^2, \quad P_{Aa} = 2pq, \quad P_{aa} = q^2 \tag{1}$$

W symulacji pomijamy mutacje, migracje i dryf genetyczny. Powyższe proporcje opisują genotypy **przed doborem** w każdym pokoleniu.
Suma frekwencji alleli oraz genotypów w populacji wynosi 1:

$$p + q = 1 \tag{2}$$

$$P_{AA} + P_{Aa} + P_{aa} = 1 \tag{3}$$

**Zmiana frekwencji allelu z uwzględnieniem doboru naturalnego**

- Średnie dostosowanie populacji wyraża się wzorem: $W_s = p^2 w_{AA} + 2pqw_{Aa} + q^2 w_{aa}$
- Zmiana frekwencji allelu *a* między pokoleniami: $\Delta q = \frac{pq}{W_s}\left[q(w_{aa} - w_{Aa}) - p(w_{AA} - w_{Aa})\right]$
- Aktualizacja frekwencji allelu *a* w kolejnym pokoleniu wygląda następująco: $q_{\text{nowe}} = q_{\text{stare}} + \Delta q$
- Dla allelu *A*: $p_{\text{nowe}} = 1 - q_{\text{nowe}}$

# Dobór naturalny: przebieg symulacji

1. **Dane wejściowe:** Użytkownik podaje:
   - Początkową frekwencję allelu $a$ ($q_0$) – wartość między 0 a 1.
   - Dostosowanie dla każdego genotypu ($w_{AA}$, $w_{Aa}$, $w_{aa}$) – wartości między 0 a 1.
   - Maksymalną liczbę pokoleń do symulacji.
2. **Symulacja:** Wykonaj symulację, obliczając dla każdego pokolenia:
   - Frekwencje genotypów ($P_{AA}$, $P_{Aa}$, $P_{aa}$).
   - Średnie dostosowanie populacji $W_s$. Jeśli $W_s = 0$, przerwij symulację przed dzieleniem i zgłoś brak możliwości reprodukcji.
   - Zmianę frekwencji allelu $\Delta q$.
   - Nowe frekwencje alleli $p$ i $q$.
3. **Zakończenie symulacji:** Zakończ symulację, gdy:
   - Frekwencja jednego z alleli osiągnie 0 lub 1.
   - Liczba pokoleń osiągnie maksymalną wartość.
   - Wartość bezwzględna zmiany frekwencji allelu $|\Delta q|$ będzie mniejsza od bardzo małej wartości (np. $1 \times 10^{-6}$).

Kod programu proszę przesłać na adres mailowy jacek.gatlik@agh.edu.pl